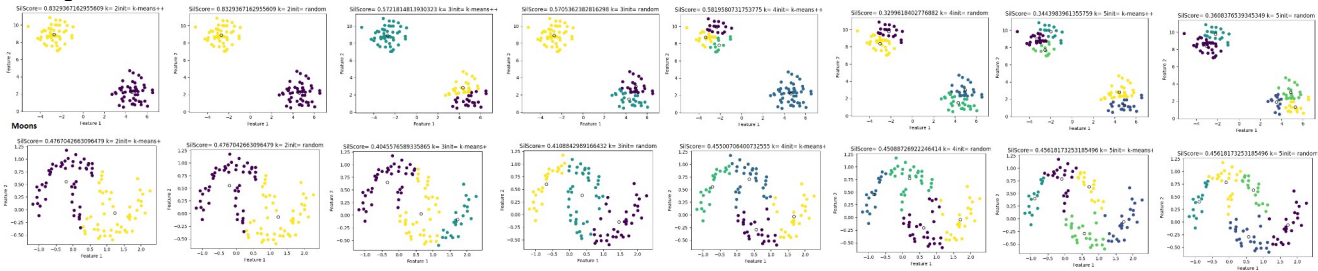


Brendan Martel
Bxm240013

Compare the effect of different initializations:



As you can see from the comparison charts above (full size image available in project files) the different initialization did not seem to have much of an effect on the quality of the cluster centers. When it came to the blobs you can see that many of the clusters are identical to each other. With the exception of k4 where the means++ was not able to split the cluster centers evenly across the clusters resulting in a lopsided classifications of that set. However this was only an issue for k4. This leads me to conclude that there was little to no difference in initialization when it came to the blobs.

When we look at the moons we can see that we have near similar results to the blobs. We see that overall the classification looks identical. This is with the exception of k3 where I feel that the random initialization gave a more appropriate classification over the means++ initialization. For this reason I would lean towards using the random initializer over the means++ when using this data.

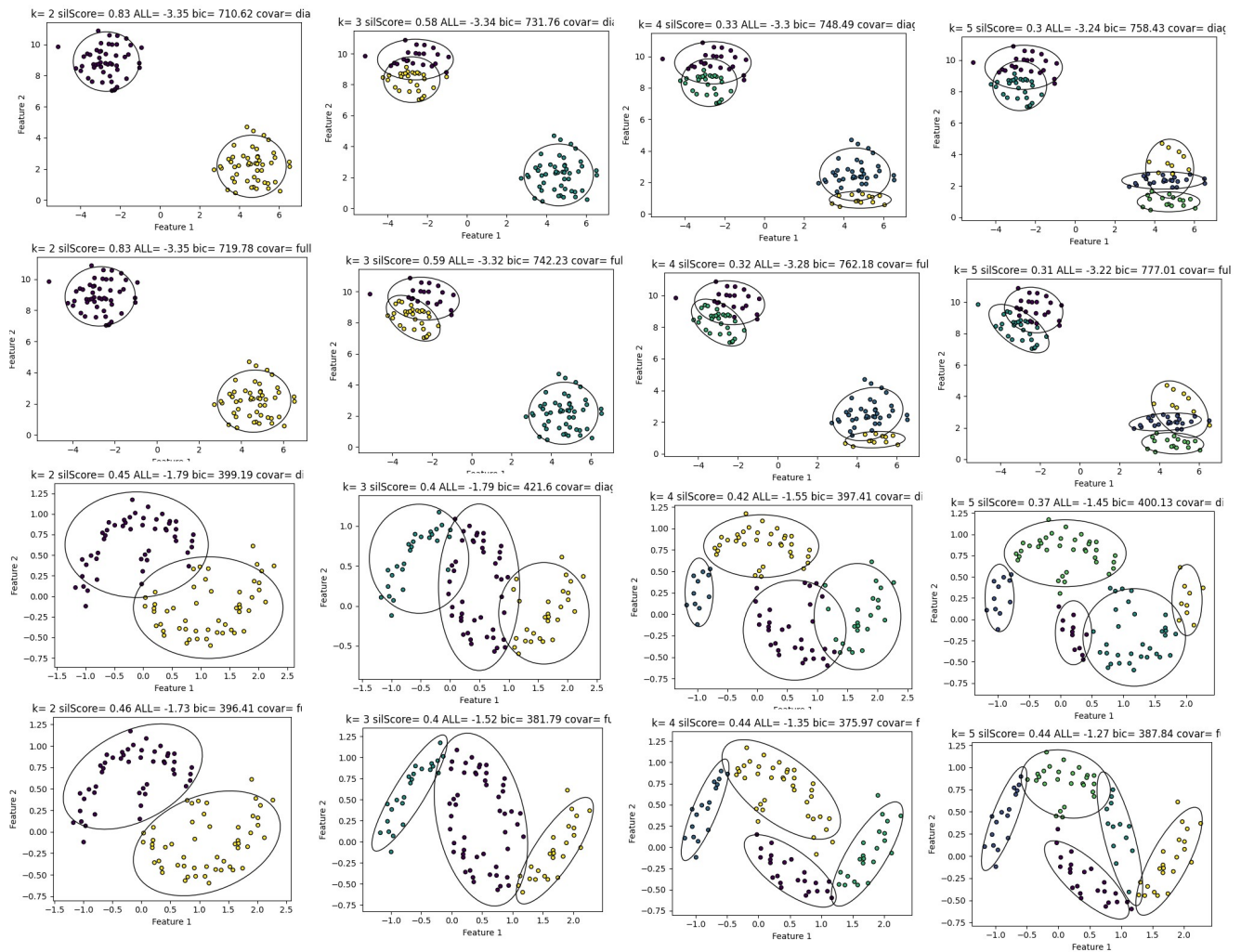
Which dataset is better suited for K-Means, and why?:

The blobs are absolutely the set that is most suited to kmeans. While yes the methods of kmeans does work on the moons and the blobs in the sense that it returns a result we can see that kmeans fails to capture the overall shape of the moons as compared to the blobs. This can be seen at all k values. For example at the smallest k value we can see that the blobs are easily captured as what they are. But the moons we see that while it does classify some of the points correctly It fails to capture about a third of the overall shape of the data. As we move towards the higher k values we can see that the classification begins to classify more in the shape of the data however we can see in areas such as the tips of the moons that data points are still being misclassified. Whereas with the blobs we can see that as k gets higher it is still able to capture the shape and boundaries of each of the blobs and bisecting the groupings in a reasonable manner.

How does the choice of k affect clustering quality?:

As discussed in the previous question when it comes the the blobs it is clear that the lower k values do a better job of capturing the shape of the data. In this case k2 would be the best but in general the best option would probably be whenever k is equal to the number of clusters that you have in your data which of course could be more difficult to find in higher dimensions. But even at k values higher then the number of clusters it still captures the shape just bisecting it into smaller and smaller pieces.

When it came to the moons it was a different story. As stated previously lower k values were unable to capture the shape of the moons. Whereas the higher k values did a better overall job at conforming to the shape of the data. However despite the capturing of the shape improving the kmeans clustering still does not posses the fidelity to accurately find the shape of the data.



Analyze how GMMs handle elliptical clusters better than K-Means, especially on non-convex datasets:

GMMs capture the shape of non circular on convex clusters much better than kmeans. First of all GMMs have a much higher fidelity than kmeans clustering. Because of the use of ellipses over just the circular distance around a center the GMMs are able to capture different shapes and patterns that kmeans may not in addition to capturing ones that Kmeans is already good at capturing. We can see this in the above comparison table (again full size available in the project files) shown above. If we compare the performance of GMM on the blobs we can see that it performs similarly to the kmeans in the previous figure. However it is also able to capture the even split on k4 unlike some initialization of the kmeans.

When it came to the moons GMMs far outperformed the kmeans in classifying the points. Again this goes back to the use of the ellipsis rather than just a circular distance. The shape provides us with better fidelity to fit things that arent perfectly separable into circles. In addition to this some initialization also give us the ability to tilt the ellipsis giving us even more fidelity. This is why on the moons kmeans is blown out of the water by the performance of the GMMs especially the full initialization. As we have established the dataset is not separable into circular chunks and is an irregular shape. We can see that at higher k values that GMM is able to capture the shape of the moons for the most part outperforming kmeans clustering.

In what scenarios does EM outperform K-Means?:

As discussed in the previous section I believe that EM outperforms kmeans in all scenarios that the data is not perfectly separable into circular chunks. I believe this to be many if not most scenarios. As discussed above I believe this to be because of the additional fidelity provided by EM allowing it to perform better than kmeans on both circularly separable and non circularly separable shapes.

How does the covariance type influence cluster shape and separation?:

Full captured the shape of the data significantly better than the diagonal covariance. This is because it allowed the ellipsis to rotate allowing them to capture data with an even higher fidelity than being stuck on two axis. This gives the full covariance a better setup to capture the shape of irregular data. We can see this in action in the above figure especially at the higher k values where EM shines. We can see this the diagonal covariance manages to capture the majority of the shape but still has some miscategorizations. While this is better than something like kmeans we can still capture the overall shape of the of the data better using the full covariance. The full covariance has fewer misclassifications but more importantly we can see that the ovals capture a better picture of the overall data shape when compared to all of the other things we have tried.

Kmenas – Blobs – silScores				Kmeans – Moons – silScores			
K	K-means++	random		K	K-means++	random	
2	0.83	0.83		2	0.47	0.47	
3	0.57	0.57		3	0.4	0.41	
4	0.58	0.32		4	0.46	0.45	
5	0.34	0.36		5	0.46	0.46	
GMM – Blobs – silScores				GMM – Blobs – silScores			
K	diagonal	full		K	diagonal	full	
2	0.83	0.83		2	0.45	0.56	
3	0.58	0.59		3	0.4	0.4	
4	0.33	0.32		4	0.42	0.44	
5	0.3	0.31		5	0.37	0.44	
GMM – Blobs – ALL				GMM – Blobs – ALL			
K	diagonal	full		K	diagonal	full	
2	-3.35	-3.34		2	-1.79	-1.73	
3	-3.34	-3.32		3	-1.79	-1.52	
4	-3.3	-3.28		4	-1.55	-1.35	
5	-3.24	-3.22		5	-1.45	-1.27	